

Communication Methods in Digital Systems Lab 3

AXI Bus - Introduction

Introduction

This laboratory is intended to illustrate the basic properties of the AXI Bus system. The experiments will be started from the AXI-Lite protocol to gain knowledge and experience in data transfers using the multipath bus system.

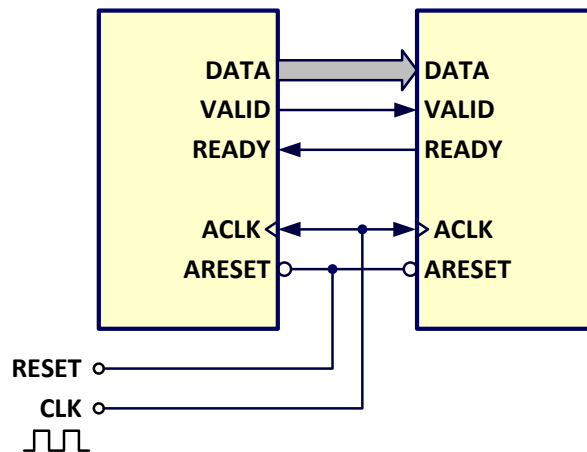


Fig. 1. The data connection in AXI

It is essential to get acquainted with the synchronous handshake protocol. In opposite to the asynchronous handshake method there is a common clock for the controller and device sides. This changes the timing of data passing as well as signaling the readiness of data source and target. Note that depending on the channel direction transmitting and receiving side can be either the controller or the target device.

The handshake process is based on the VALID and READY lines. The VALID line notifies that there are present a valid data on the bus. The data provider must wait until the data are accepted by the data recipient using READY signal. The acknowledge cycle is based on mutual activity of VALID and READY signals. READY and VALID lines must not to be active longer than one clock cycle. Different mutual dependencies of signals are considered in following figures.

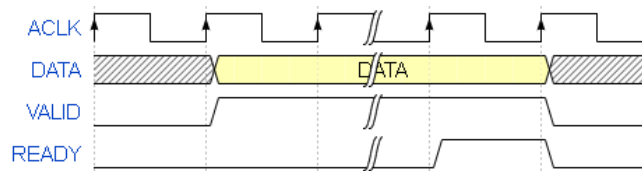


Fig. 2. Handshake signal flow with READY after VALID

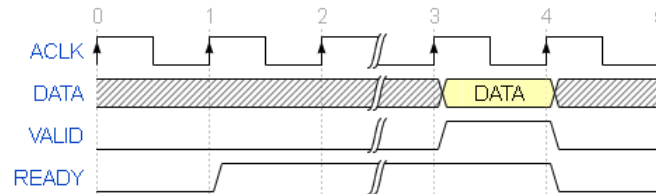


Fig. 3. Handshake signal flow with READY before VALID

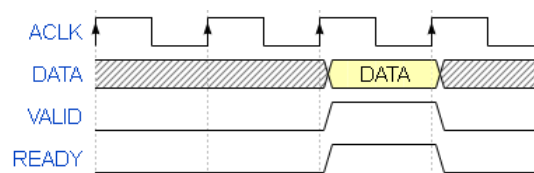


Fig. 4. Handshake signal flow with simultaneous READY and VALID assertion

There can be distinguished 3 different cases of mutual VALID and READY dependencies. Fig. 2 illustrates a typical flow where VALID is asserted before READY. The data recipient acknowledges the transaction after data validity is notified. In case when the recipient is ready for data transfer it can assert a READY line before the sender delivers the data confirming it by asserting the VALID line (Fig. 3). Finally, in the case of a burst transaction, it is possible to have a situation when the READY and VALID are asserted in the same cycle as shown in Fig. 4.

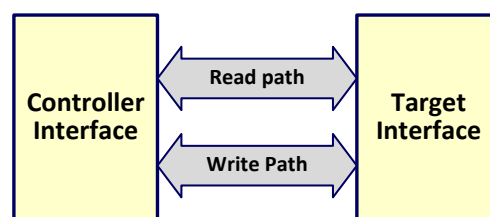


Fig. 5. The AXI Read and Write paths

The bus system implements independent read and write paths that enable simultaneous bidirectional data flow.

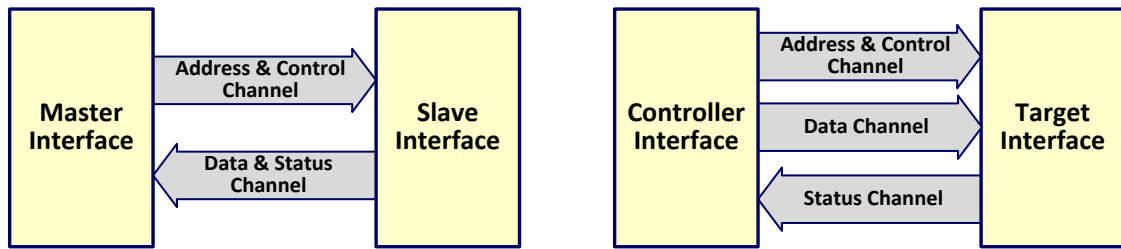


Fig. 6. The AXI Read and Write channel details

The read and the write paths use address and control channels to initialize transactions. The data channel is used for data exchange and finally, the transaction status is sent back from the device to the controller. The write channel implements a separate status channel. Delivering the operation status is performed by the device after delivery complete data package. In the read channel this the status is transferred with last data item passed from the device.

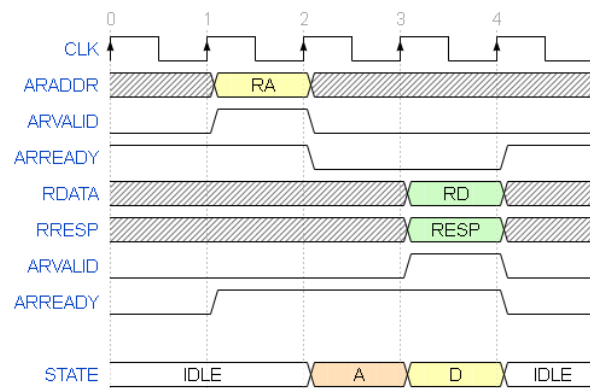


Fig. 7. AXI Lite read transaction

The data read transactions are shown in Fig. 7. The controller requests the data from particular address during the read transaction. In order to speed up the operation device asserts the ARREADY line signaling an immediate acceptance of arriving request. It should be note that the time distance between request and response is not specified. It is obvious that the device should complete the request as soon as possible. Due to (potential) hierarchical architecture of the bus system the response can be delivered after several cycles.

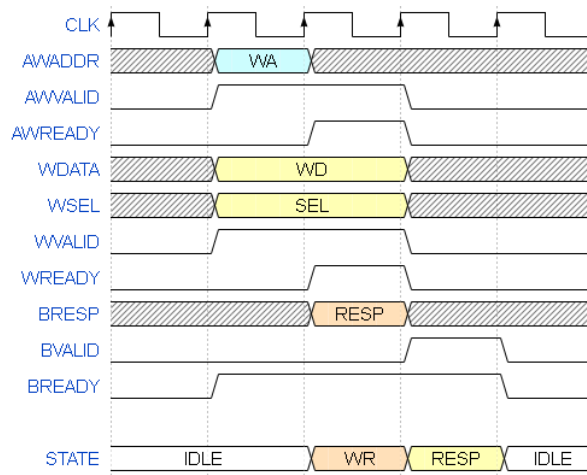


Fig. 8. AXI Lite write transaction

The data write transaction is shown in Fig. 8.

Design tasks for the AXI Lite interface

In this laboratory we are going to design a Bus Functional Model of master controller that enables to model the complete data read or write transfer upon the user request by calling a task. The opposite side will be a synthesizable implementation of device equipped with the AXI Lite interface. This will allow to observe the problems of developing particular models.

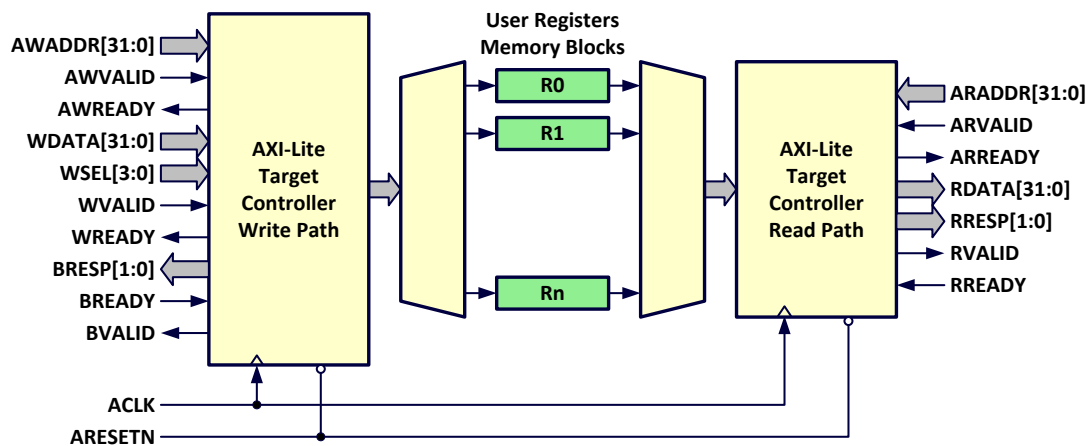


Fig. 9. AXI Lite device block diagram.

The goal of the design is development of the AXI Lite device that block diagram is shown in Fig. 9. Due to complexity of the interface it is almost impossible to design the device without testing it. The device is connected to the complex system that model is not always available. The whole system model is very complex and it is questionable if it is really necessary for correctness of the design. In order to simplify

modelling process the Bus functional models are used to model bus traffic and determine the correctness of operation. The BFM's are pure behavioral models.

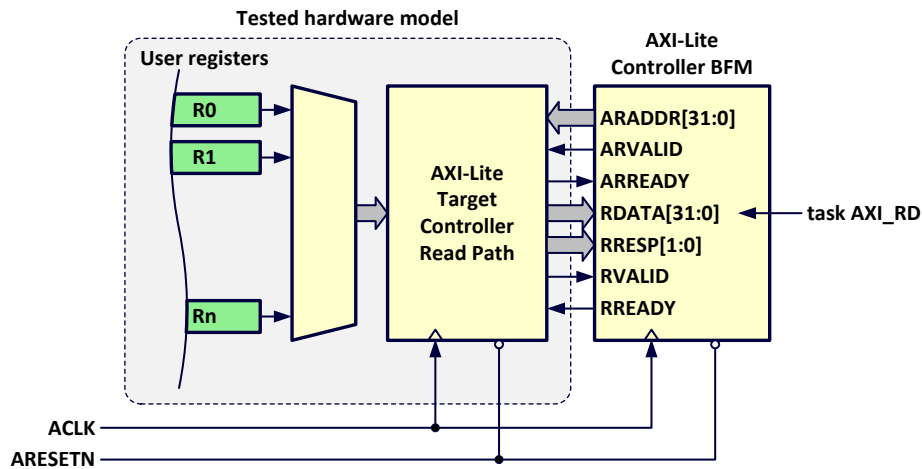


Fig. 10. AXI Lite controller BFM model and the device simulation model.

The connection of the BFM modelling the read path is shown in Fig. 10. The RD task enables triggering the data read transaction:

```
task RD;
input [31:0] ADDR;
output [31:0] DATA;
output [1:0] RESP;
begin
    //...
end
endtask
```

The task accepts the transaction address and responds with the received data and transaction status. The task is called asynchronously to the controller state and needs to implement synchronization mechanisms to correctly drive the interface controller.

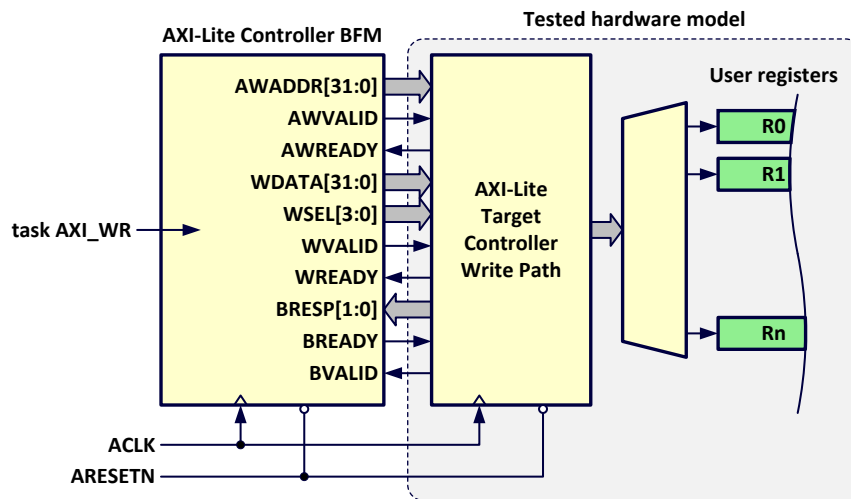


Fig. 11. AXI Lite controller BFM model and the device simulation model.

Similarly to the read path, the write path functional model is constructed. The WR task transfers data packet and retrus the ransfer status. This can be illustrated with following code snippet:

```
task WR;
input [31:0] ADDR;
input [31:0] DATA;
input [3:0] STRB;
output [1:0] BRESP;
begin
    //...
end
endtask
```

Tasks

1. Implement BFMs for read and write data channels of AXI Lite protocol. Assure reporting transaction status to the console
2. Design an input and output port with an AXI-Lite interface. Enable registers for: direct port data write, bit set, bit clear, and bit toggle. Implement respective test environment to prove the operation correctness.
3. Design a timer with a presetable interval of interrupt notification. It should be implemented as a down counter with a synchronous load of preset value when zero is reached. Reaching zero value should trigger the notification. If the interrupt enable bit is set it should assert the IRQ line. The interrupt notification is cleared by writing 1 to the control register in the position of the notification flag. The timer should have the following registers:

- a. Control register containing: RUN, STOP, and ZD_ACK (zero detect acknowledge) bits. Writing 1 to respective positions enables the functionality. To resolve the START/STOP conflict the STOP has priority over START. The START of the timer is allowed only if the Count value is > 255 . In another case, the Timer remains in the stopped state.
- b. Status register contains the RUN and ZD (zero detect)
- c. Count value register with RW access. Can be updated provided the timer is in stop mode. When the timer is running changes are not allowed. Return in response SLVERR AXI status
- d. The counter register of the timer should deliver the current count value for read. The content should be captured in the register in case of delayed arrival of READY notification.